A Discussion: Function Vectors vs Semantic Vector Arithmetic

In this appendix we discuss the experimental support for our characterization of function vectors in more detail, in particular the assertion that function vectors are acting in a way that is distinct from semantic vector arithmetic on word embeddings.

The use of vector addition to induce a mapping is familiar within semantic embedding spaces: the vector algebra of *semantic vector offsets* has been observed in many settings; for example, word embedding vector arithmetic was clearly described by Mikolov et al. (2013), and has been observed in other neural word representations including transformers (recently, Merullo et al., 2023). In recent examinations of internal transformer states, Geva et al. (2022) and Dar et al. (2023) have suggested that many internal transformer calculations can be understood in terms of such word vector arithmetic. Therefore one of our main underlying research questions is whether our function vectors should be described as triggers for a nontrivial function, or whether, more simply (as would be suggested by Dar et al.), they could be thought of as just ordinary semantic vector offsets that induce a trivial mapping between related words by adding an offset to a rich word embedding.

Our investigation of the possibility of a distinct and localized representation for tasks is similar to the studies of task encodings in RNNs by Lake & Baroni (2018) and Hill et al. (2018), as well as the studies of linguistic attribute encodings by Clark et al. (2019), but our focus on causal effects rather than correlations allows us to explicitly extract and test the computational roles of vector representations of functions, which leads to several new lines of evidence.

The main paper contains three pieces of experimental evidence that support the conclusion that function vectors are different from semantic vector offsets of word embeddings, and that they trigger nontrivial functions:

1. Function vectors can implement complex mappings, including cyclic mappings such as *antonyms* that cannot be semantic vector offsets.
2. Function vector causal effects cannot be recovered from the target output vocabulary alone; they carry some other information.
3. Function vector activity is mediated by mid-layer nonlinearities (i.e., they trigger nonlinear computations), since they have near-zero causal effect at late layers.

We discuss each of these lines of evidence in more detail here.

Cyclic Tasks Cannot be Semantic Vector Offsets. The first task analyzed in the paper is the *antonym* task. Because the antonym task is cyclic, it is a simple counterexample to the possibility that function vectors are just semantic vector offsets of language model word embeddings.

Suppose there were an 'antonym' vector offset v_a such that adding it to a word embedding w produces w', the embedding of the antonym of w (i.e. $w + v_a = \text{antonym}(w) = w'$). An example of this might be $\texttt{vec(big)} + v_a = \texttt{vec(small)}$. But then: if w and w' are antonyms of each other, the relationship holds both ways. That means that the antonym offset vector v_a would have to satisfy both $w + v_a = w'$ and $w' + v_a = w$, which can only happen if $v_a = 0$, implying $w = w'$ and creating a contradiction to the assumption that w and w' are antonyms, distinct from each other. Thus there could be no constant vector offset that would properly model the antonym task. The same argument excludes semantic vector offsets for any cyclic function. We evaluate additional cyclic tasks in Appendix L.

Since we are able to find a constant antonym function vector v_t that, when added to the transformer, does cause cyclic behavior, we conclude that the action of v_t is a new phenomenon. Function vectors act in a way different from simple semantic vector offsets.

Function Vectors Contain Information Beyond Output Vocabulary. Not every function is cyclic, but the vector offset hypothesis can be tested by examining word embeddings. Following the reasoning of Geva et al. (2022); Dar et al. (2023), one way to potentially implement a semantic vector offset is to promote a certain subset of tokens that correspond to a particular semantic concept (i.e. capital cities, past-tense verbs, etc.). Function vectors do show some evidence of acting in this way: when decoding function vectors directly to the model's vocabulary space we often see that the tokens

with the highest probabilities are words that are part of its task's output vocabulary (see Table 5 and Table 19).

To determine whether it is the vocabulary itself that contributes to the function vector's performance, in Section 3.2 we construct another vector (via optimization) that decodes to the same vocabulary distribution and measure its performance when used as a "function vector". In that experiment we create reconstructions $\hat{v}_{t100}$ and $\hat{v}_{t\text{all}}$ that encode the same decoder vocabulary as v_t (the near-zero KL divergences show that the reconstructions of the decoder vocabulary are near-perfect). Yet the performance of the reconstructions when used as FV is poor when the top 100 words are matched, and still lower than v_t even when the distribution over the full vocabulary is reconstructed.

That experiment reveals that while function vectors do often encode words contained in the output space of the task they are extracted from, simply adding a vector that boosts those same words by the same amounts is not enough to recover its full performance—though if enough words are included, in some cases a fraction of performance is recovered. Our measurements suggest that while *part* of the role of a function vector may act similarly to a semantic vector offset, the ability for function vectors to produce nontrivial task behavior arises from other essential information in the vector beyond just a simple word embedding vocabulary-based offset. See Appendix N for a related experiment.

Function Vectors' Causal Effects are Near-Zero at Late Layers. Across the set of tasks and models we evaluate, there is a common pattern of causal effects that arises when adding a function vector to different layers. The highest causal effects are achieved when adding the function vector at early and middle layers of the network, with a sharp drop in performance to near-zero at the later layers of the network (Figure 4, Figure 26, Figure 14, Figure 16). Interestingly, FV causal effects are strongest in largest models, yet the cliff to near-zero causal effects is also sharpest for the largest models (Figure 16).

If the action were linear and created by a word embedding vector offset, then the residual stream would transport the action equally well at any layer including the last layers. Thus the pattern of near-zero causal effects at later layers suggests that the action of the function vector is not acting directly on the word embedding, but rather that it is mediated by some *nonlinear* computations in the middle layers of the network that are essential to the performance of the task. This mediation is evidence that the function vector activates mid-layer components that execute the task, rather than fully executing the task itself.

This pattern is in contrast to the vector arithmetic described in Merullo et al. (2023), that are most effective at later layers of the network and have little to no causal effect at early layers of the network; those offsets more closely resemble semantic vector offsets of word embeddings.

Function Vectors Therefore Represent Functions In summary, the three lines of evidence lead us conclude that the vectors v_t should not be seen as simple word embeddings, nor trivial offsets or differences of embeddings, nor simple averages of word embeddings over vocabularies of words to be boosted. These characteristics distinguish function vectors from from many linguistic concepts that can be viewed as probability distributions over words. Rather, the evidence suggests that the vectors we have identified act in a way that is distinct from literal token embedding offsets. Instead they directly represent and trigger nonlinear execution of abstract functions.

This finding is surprising since the transformer is trained on a word-prediction task; it would be expected that they should learn representations that can be expressed in terms of adjustments to word probabilities as observed by Geva et al. (2022); Dar et al. (2023). Our evidence indicates a different kind of representation: we find that transformers learn a compact, concrete, and causal vector representation of higher-level functional concepts that cannot be reduced to a probability vector over a set of words.

Thus we come to the conclusion that the vectors v_t are references to functions, and we call them *function vectors*.

B ATTENTION OUTPUTS ARE INDEPENDENT AND ADDITIVE

In this section we define our formulation of attention notation $a_{\ell j}$ in detail, relating our notation to the original formulation of Vaswani et al. (2017) via the framework of Elhage et al. (2021).

B.1 EXPRESSING ATTENTION $a_{\ell j}$ IN THE RESIDUAL STREAM SPACE

The transformer architecture as introduced by Vaswani et al. (2017) has a multihead attention mechanism. They describe it in terms of a concatenation procedure for performing the attention function for several "heads" $\text{head}_j \in \mathbb{R}^{d_v}$ $(j \leq J)$ at each layer, all in parallel. Equation 9 reproduces the relevant equation from page 5 of Vaswani et al.:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, ..., \text{head}_h)W^O \tag{9}$$

$$\text{where head}_j = \text{Attention}(QW_j^Q, KW_j^K, VW_j^V) \tag{10}$$

Note that a transformer repeats this process with different weights and data at each layer; we add the layer subscript ℓ to disambiguate their notation. In the Vaswani et al. formulation, each head at a layer ℓ resides in a low dimensional space $\mathbb{R}^{d_v}$ with dimension $d_v < d$ that differs from the main hidden state *residual stream* of the transformer, which we write as $\mathbf{h}_\ell \in \mathbb{R}^d$. All the heads at the layer are concatenated and then transformed through transformation W_ℓ^O to produce the full MultiHead_ℓ attention output in $\mathbb{R}^d$.

Elhage et al. (2021) observes that the Vaswani et al. formulation is equivalent to dividing the matrix W_ℓ^O into block form $[W_{\ell 1}^O \; W_{\ell 2}^O \ldots \; W_{\ell J}^O]$, and then projecting each $\text{head}_{\ell j}$ into the residual stream space directly. In our formulation of $a_{\ell j}$ in Section 2.2, we adopt this view. The attention head output $a_{\ell j}$ can be defined in terms of the notation of Vaswani et al. and Elhage et al. as follows:

$$a_{\ell j} = \text{head}_{\ell j} W_{\ell j}^O \in \mathbb{R}^d \tag{11}$$

In this way the total attention contribution at layer ℓ is the sum of the attention output of each individual head, and these all reside directly in the $\mathbb{R}^d$ space of the residual stream:

$$\text{MultiHead}_\ell(Q_\ell, K_\ell, V_\ell) = \sum_{j \leq J} a_{\ell j} \in \mathbb{R}^d \tag{12}$$

While the left and right-hand sides of equation 12 are computationally equivalent definitions of attention, the "independently additive" form of attention on the right-hand side allows us to see the contributions of individual attention heads more clearly.

B.2 ADDING FUNCTION VECTORS TO A LAYER

Using this formulation of attention $a_{\ell j}$, we return to our notation as defined in section 2.2 to understand what we mean when we say we add a function vector to a layer ℓ.

We focus on the hidden state residual stream at the final token of a given prompt. Recall that the ordinary operation of a transformer defines the hidden state

$$\mathbf{h}_\ell = \mathbf{h}_{\ell-1} + m_\ell + \sum_{j \leq J} a_{\ell j} \tag{13}$$

This recursive residual structure forms a telescoping sum that creates a common vector space for nearby layers; for example Zhao et al. (2021) has observed that adjacent layers of a transformer can be swapped with little change. Thus it is meaningful to collect, average, and sum attention head outputs $a_{\ell j} \in \mathbb{R}^d$ among nearby layers, and that observation inspires our definition of a function vector v_t as average attention head values for a selection of relevant heads $\bar{a}_{\ell j}$ (equation 5). See Appendix M for an analysis of an alternative formulation that does not swap layers.

Since the function vector also resides in the residual stream space of $\mathbf{h}_\ell \in \mathbb{R}^d$, when we add a function vector v_t to the hidden state of layer ℓ, we can therefore add it to the hidden state just as attention is added; we write the updated hidden state $\mathbf{h}'_\ell$ as

$$\mathbf{h}'_\ell = \mathbf{h}_{\ell-1} + m_\ell + \sum_{j \leq J} a_{\ell j} + v_t \tag{14}$$

This could also be written as simply $\mathbf{h}'_\ell = \mathbf{h}_\ell + v_t$.